

Algorithm

STL 算法

STL 提供了大约 100 个实现算法的模版函数，基本都包含在 `<algorithm>` 之中，还有一部分包含在 `<numeric>` 和 `<functional>`。完备的函数列表请 [参见参考手册](#)，排序相关的可以参考 [排序内容的对应页面](#)。

- **find**：顺序查找。`find(v.begin(), v.end(), value)`，其中 `value` 为需要查找的值。
- **reverse**：翻转数组、字符串。`reverse(v.begin(), v.end())` 或 `reverse(a + begin, a + end)`。
- **unique**：去除容器中相邻的重复元素。`unique(ForwardIterator first, ForwardIterator last)`，返回值为指向去重后容器结尾的迭代器，原容器大小不变。与 `sort` 结合使用可以实现完整容器去重。
- **random_shuffle**：随机地打乱数组。`random_shuffle(v.begin(), v.end())` 或 `random_shuffle(v + begin, v + end)`。

▼ `random_shuffle` 函数在最新 C++ 标准中已被移除

`random_shuffle` 自 C++14 起被弃用，C++17 起被移除。

在 C++11 以及更新的标准中，您可以使用 `shuffle` 函数代替原来的 `random_shuffle`。使用方法为 `shuffle(v.begin(), v.end(), rng)`（最后一个参数传入的是使用的随机数生成器，一般情况使用以真随机数生成器 [random_device](#) 播种的梅森旋转伪随机数生成器 [mt19937](#)）。

```
1 // #include <algorithm>
2 std::mt19937 rng(std::random_device{}());
3 std::shuffle(v.begin(), v.end(), rng);
```

- **sort**：排序。`sort(v.begin(), v.end(), cmp)` 或 `sort(a + begin, a + end, cmp)`，其中 `end` 是排序的数组最后一个元素的后一位，`cmp` 为自定义的比较函数。
- **stable_sort**：稳定排序，用法同 `sort()`。
- **nth_element**：按指定范围进行分类，即找出序列中第 `n` 大的元素，使其左边均为小于它的数，右边均为大于它的数。`nth_element(v.begin(), v.begin() + n, v.end(), cmp)` 或 `nth_element(a + begin, a + begin + n, a + end, cmp)`。
- **binary_search**：二分查找。`binary_search(v.begin(), v.end(), value)`，其中 `value` 为需要查找的值。
- **merge**：将两个（已排序的）序列有序合并到第三个序列的插入迭代器上。`merge(v1.begin(), v1.end(), v2.begin(), v2.end(), back_inserter(v3))`。
- **inplace_merge**：将两个（已按小于运算符排序的）：`[first, middle)`，`[middle, last)` 范围原地合并为一个有序序列。`inplace_merge(v.begin(), v.begin() + middle, v.end())`。
- **lower_bound**：在一个有序序列中进行二分查找，返回指向第一个大于等于 `x` 的元素的位置的迭代器。如果不存在这样的元素，则返回尾迭代器。`lower_bound(v.begin(), v.end(), x)`。
- **upper_bound**：在一个有序序列中进行二分查找，返回指向第一个大于 `x` 的元素的位置的迭代器。如果不存在这样的元素，则返回尾迭代器。`upper_bound(v.begin(), v.end(), x)`。

▼ `lower_bound` 和 `upper_bound` 的时间复杂度

在一般的数组里，这两个函数的时间复杂度均为 $O(\log n)$ ，但在 `set` 等关联式容器中，直接调用

`lower_bound(s.begin(), s.end(), val)` 的时间复杂度是 $O(\log n)$ 的。

`set` 等关联式容器中已经封装了 `lower_bound` 等函数（像 `s.lower_bound(val)` 这样），这样调用的时间复杂度是 $O(\log n)$ 的。

- `next_permutation`：将当前排列更改为全排列中的下一个排列。如果当前排列已经是全排列中的最后一个排列（元素完全从大到小排列），函数返回 `false` 并将排列更改为全排列中的第一个排列（元素完全从小到大排列）；否则，函数返回 `true`。`next_permutation(v.begin(), v.end())` 或 `next_permutation(v + begin, v + end)`。
- `prev_permutation`：将当前排列更改为全排列中的上一个排列。用法同 `next_permutation`。
- `partial_sum`：求前缀和。设源容器为 `src`，目标容器为 `dst`，则令 `partial_sum(src.begin(), src.end(), back_inserter(dst))`。

使用样例

- 使用 `next_permutation` 生成 到 的全排列。例题：[Luogu P1706 全排列问题](#)

▼ 实现

```
1 int N = 9, a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
2 do {
3     for (int i = 0; i < N; i++) cout << a[i] << " ";
4     cout << endl;
5 } while (next_permutation(a, a + N));
```

- 使用 `lower_bound` 与 `upper_bound` 查找有序数组 中小于，等于，大于 元素的分界线。

▼ 实现

```
1 int N = 10, a[] = {1, 1, 2, 4, 5, 5, 7, 7, 9, 9}, x = 5;
2 int i = lower_bound(a, a + N, x) - a, j = upper_bound(a, a + N, x) - a;
3 // a[0] ~ a[i - 1] 为小于x的元素, a[i] ~ a[j - 1] 为等于x的元素,
4 // a[j] ~ a[N - 1] 为大于x的元素
5 cout << i << " " << j << endl;
```

- 使用 `partial_sum` 求解 中元素的前缀和，并存储于 中。

▼ 实现

```
1 vector<int> src = {1, 2, 3, 4, 5}, dst;
2 // 求解src中元素的前缀和, dst[i] = src[0] + ... + src[i]
3 // back_inserter 函数作用在 dst 容器上, 提供一个迭代器
4 partial_sum(src.begin(), src.end(), back_inserter(dst));
5 for (unsigned int i = 0; i < dst.size(); i++) cout << dst[i] << " ";
```

- 使用 `lower_bound` 查找有序数组 中最接近 的元素。例题：[UVa10487 Closest Sums](#)

▼ 实现

```
1 int N = 10, a[] = {1, 1, 2, 4, 5, 5, 8, 8, 9, 9}, x = 6;
2 // lower_bound将返回a中第一个大于等于x的元素的地址, 计算出的i为其下标
3 int i = lower_bound(a, a + N, x) - a;
4 // 在以下两种情况下, a[i] (a中第一个大于等于x的元素) 即为答案:
5 // 1. a中最小的元素都大于等于x;
6 // 2. a中存在大于等于x的元素, 且第一个大于等于x的元素 (a[i])
7 // 相比于第一个小于x的元素 (a[i - 1]) 更接近x;
8 // 否则, a[i - 1] (a中第一个小于x的元素) 即为答案
9 if (i == 0 || (i < N && a[i] - x < x - a[i - 1]))
10     cout << a[i];
11 else
12     cout << a[i - 1];
```

- 使用 `sort` 与 `unique` 查找数组 中第 小的值（注意：重复出现的值仅算一次，因此本题不是求解第 小的元素）。例题：[Luogu P1138 第 k 小整数](#)

▼ 实现

```
1 int N = 10, a[] = {1, 3, 3, 7, 2, 5, 1, 2, 4, 6}, k = 3;
2 sort(a, a + N);
3 // unique将返回去重之后数组最后一个元素之后的地址，计算出的cnt为去重后数组的长度
4 int cnt = unique(a, a + N) - a;
5 cout << a[k - 1];
```

本页面最近更新：2024/12/1 22:40:19, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[cmpute](#), [StudyingFather](#), [aotenjou](#), [chentingyu.20](#), [Enter-tainer](#), [gi-b716](#), [GoodCoder666](#), [HeRaNO](#), [ImpleLee](#), [lr1d](#), [Menci](#), [opsiff](#), [ouuan](#), [sbofgayschool](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用